Name: M S GOWRISH SHANKAR REDDY
Reg no: 192424270
Date:24/8/26

1. Detrending

Given scenario

At a tourism board, an analyst was asked by the Tourism Director to visualize monthly visitor count across 8 destinations over time. Evaluate whether an underlying trend is masking a real cyclic/seasonal pattern that only becomes visible after detrending.

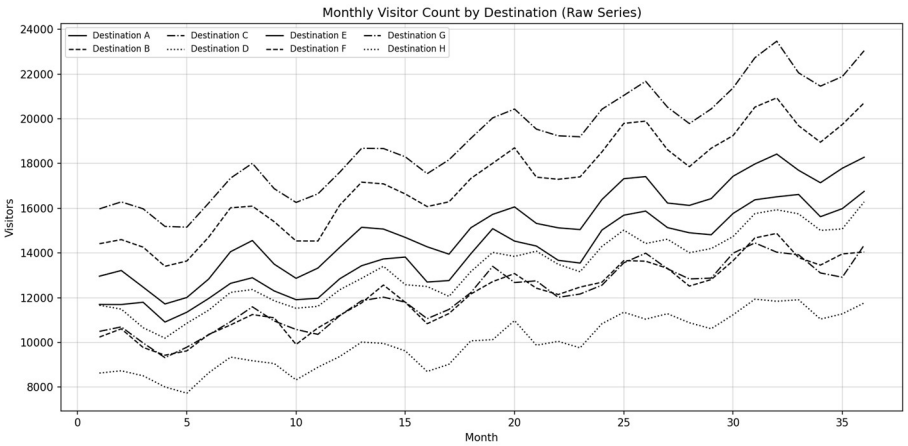
Code:

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt

data = pd.read_csv("tourism_visitors.csv")
destinations = data["Destination"].unique()
linestyles = ['-', '--', '-.', ':', '-', '--', '-.', ':']

plt.figure(figsize=(12, 6))
for dest, ls in zip(destinations, linestyles):
    d = data[data["Destination"] == dest]
    plt.plot(d["Month"], d["Visitors"], color="black",
             linestyle=ls, linewidth=1.2, label=dest)
plt.title("Monthly Visitor Count by Destination (Raw Series)")
plt.xlabel("Month"); plt.ylabel("Visitors")
plt.legend(ncol=4, fontsize=8)
plt.grid(alpha=0.3, color="gray")
plt.tight_layout()
plt.show()
```

Output:



2. Forecasting Trends

Given scenario

At the tourism board, the analyst was asked to forecast future monthly visitor counts. Evaluate whether uncertainty is honestly communicated, and whether the forecast horizon is reasonable given the length and noise of the historical data.

Code:

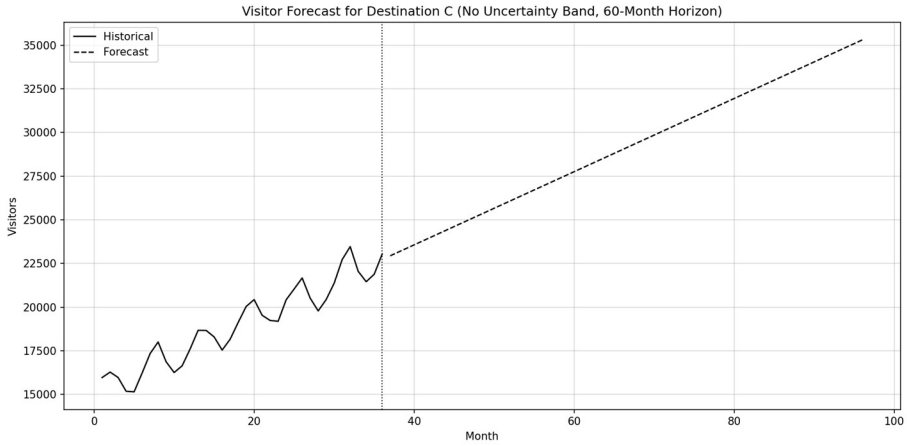
```
dest = "Destination C"
d = data[data["Destination"] == dest].sort_values("Month")
x, y = d["Month"].values, d["Visitors"].values
slope, intercept = np.polyfit(x, y, 1)

future_bad = np.arange(37, 97)
forecast_bad = slope * future_bad + intercept

plt.figure(figsize=(12, 6))
plt.plot(x, y, color="black", label="Historical")
```

```
plt.plot(future_bad, forecast_bad, color="black", linestyle="--", label="Forecast")
plt.axvline(36, linestyle=":", color="black")
plt.title(f"Visitor Forecast for {dest} (No Band, 60-Month Horizon)")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



3. Individual Time Series

Given scenario

At an energy utility, an analyst was asked by the Grid Operations Head to visualize daily energy output (MWh) across 6 power plants over time. Evaluate whether the axis scale, gridlines, and labelling let a viewer correctly read specific values, and whether the chart honestly shows the true variability.

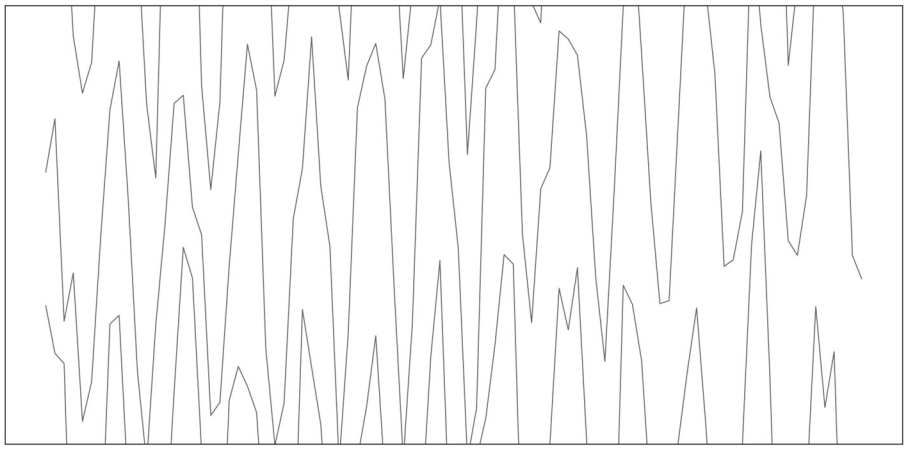
Code:

```
data = pd.read_csv("energy_output.csv")
plants = data["Plant"].unique()


plt.figure(figsize=(12, 6))
for plant in plants:
    d = data[data["Plant"] == plant]
    plt.plot(d["Day"], d["OutputMWh"], color="black", alpha=0.6)
plt.ylim(400, 460)
plt.xticks([]); plt.yticks([])
plt.tight_layout()
```

```
plt.show()
```

Output:



4. Seasonal Decomposition (STL)

Given scenario

At the energy utility, an analyst was asked to determine whether the daily output data can be separated into trend, seasonal, and residual components. Evaluate whether trend and seasonal components are actually separated and quantified rather than guessed, and whether the residual was checked for leftover pattern.

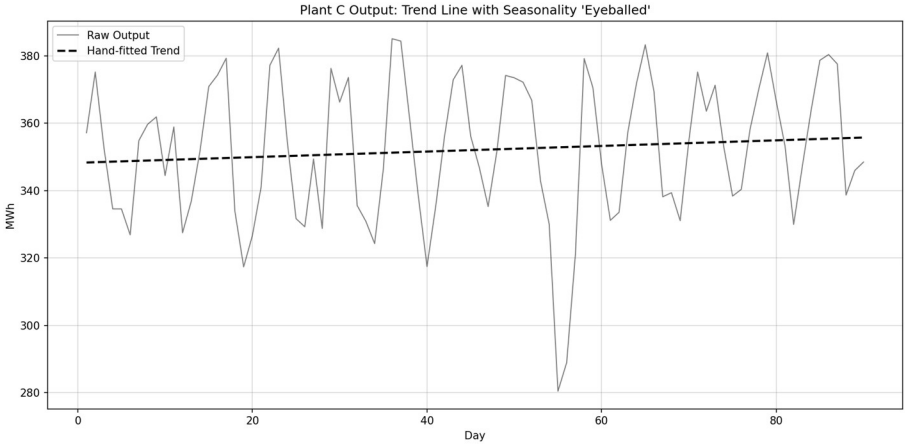
Code:

```
plant = data[data["Plant"] == "Plant
C"].sort_values("Day").reset_index(drop=True)
y = plant["OutputMWh"].values
d = plant["Day"].values
slope, intercept = np.polyfit(d, y, 1)
eyeball_trend = slope * d + intercept


plt.figure(figsize=(12, 6))
plt.plot(d, y, color="black", alpha=0.5, label="Raw Output")
plt.plot(d, eyeball_trend, color="black", linewidth=2, linestyle="--",
        label="Hand-fitted Trend")
plt.title("Plant C Output: Trend Line with Seasonality 'Eyeballed'")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



5. Moving Averages

Given scenario

At the energy utility, an analyst was asked to visualize daily energy output using moving averages. Evaluate whether the window size is disclosed, whether the resulting lag is acceptable, and whether one window suffices or multiple windows are needed.

Code:

```
plant = data[data["Plant"] == "Plant
F"].sort_values("Day").reset_index(drop=True)
plant["MA"] = plant["OutputMWh"].rolling(30).mean()


plt.figure(figsize=(12, 6))
plt.plot(plant["Day"], plant["OutputMWh"], color="black", alpha=0.4,
        label="Actual Output")
plt.plot(plant["Day"], plant["MA"], color="black", linewidth=2,
        label="Moving Average")
plt.title("Plant F Daily Energy Output with Moving Average")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```

Output: